

POLITECNICO
MILANO 1863

Best Bike Paths – DD Document

DD document for the
Software Engineering 2 course assignment

Authors: **Alessandro Burato, Gabriele Crimella, Davide Faccini**

Teacher: Elisabetta Di Nitto
Academic Year: 2025-26

Contents

Contents **i**

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Definitions, Acronyms, Abbreviations . . . 1
1.4 Reference Documents . . . 2
1.5 Document Structure . . . 2

2 Architectural Design **3**
2.1 Overview . . . 3
2.2 Component View . . . 3
2.2.1 Mobile . . . 3
2.2.2 Backend . . . 3
2.3 Deployment View . . . 3
2.4 Runtime View . . . 4
2.5 Component Interfaces . . . 4
2.6 Styles and Patterns . . . 5
2.7 Other Design Decisions . . . 6

3 User Interface Design **7**

4 Requirements Traceability **9**

5 Implementation, Integration and Test Plan **11**
5.1 Overview and Implementation Plan . . . 11
5.2 Features identification . . . 11
5.3 Integration Strategy . . . 11
5.4 System Testing Strategy . . . 11

6 Effort Spent **13**

7 References **15**

List of Figures **17**

List of Tables **19**

1 Introduction

1.1. Purpose

The purpose of this document is to serve as a detailed descriptor of the BBP project, to be used by actual developers and as an actual agreement between customer and contractors: it will contain detailed informations about the architecture of the system, its full functionalities, constraints and it will allow the customer to validate the integrity and completeness of said system. With this DD, the customer will also be able to validate all of the requirements, to verify if the system finally meets the expectations.

1.2. Scope

The Best Bike Paths (BBP) project, initiated by a leading bike association, aims to be an advanced support system for the cycling community designed to thrive within a transport landscape dominated by cars.

The core idea is to foster a collaborative digital environment where cyclists can actively record, consult, and publish both their own and other users' riding paths.

This platform will serve as the definitive centralized resource for finding highly recommended routes tailored precisely to the user's specific experience, preferences, and safety considerations.

Ultimately, the system aims to significantly enhance the quality of the cycling experience by providing verified, community-rated routes, thereby connecting riders and making high-quality path discovery universally accessible

1.3. Definitions, Acronyms, Abbreviations

- BBP: Best Bike Paths
- RASD: Requirements Analysis and Specification Document
- DD: Design Document
- IIT: Implementation, Integration and Test Plan

- WAF: Web Application Firewall

1.4. Reference Documents

- Specification document assignment
- IEEE standard documentation for DD
- Materials for the Software Engineering 2 course made available on WeBeep

1.5. Document Structure

The document is organized in seven different chapters that will cover every detail on the actual design of our project:

- **Chapter 1 - Introduction:** this chapter shows the aim of the doucment, what it is needed to know before to actually start the read of the whole text and restates the scope of the project.
- **Chapter 2 - Architectural Design:** this chapter shows all of the main components of the whole system, including architectural choices and patterns used.
- **Chapter 3 - User Interface Design:** this chapter shows and explains how the mockup pages we created integrate all of the implemented functionalities.
- **Chapter 4 - Requirements Traceability:** this chapter shows how we ensured that every requirement needed to the system to work is being satisfied with relative design choices.
- **Chapter 5 - Implementation, Integration and Test Plan:** this chapter shows how all of the components are implemented and their integration inside the system, also providing a test plan for all of their instances.
- **Chapter 6 - Effort Spent:** this chapter includes a table where the amount of hour of commitment of every participant is shown relative to its section.
- **Chapter 7 - References:** finally, the last chapter contains all of our document references used to better refine our DD.

2 Architectural Design

2.1. Overview

2.2. Component View

2.2.1. Mobile

2.2.2. Backend

2.3. Deployment View

In this section of the document we will be focusing on the whole deployment of BBP and the motivations behind our choices.

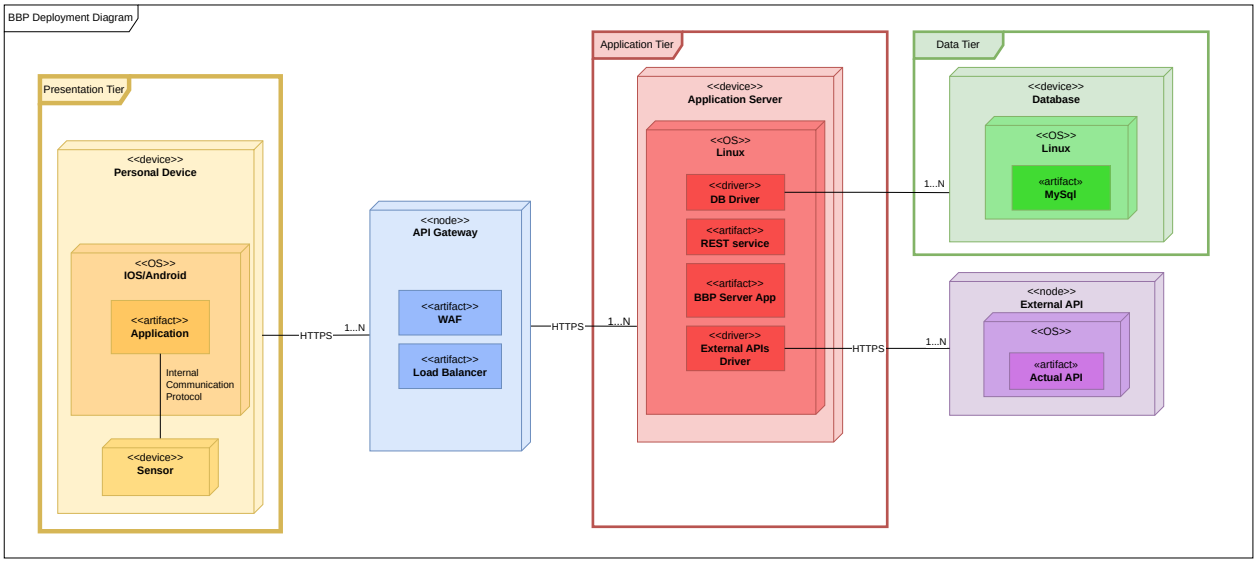


Figure 2.1: Deployment View

We divided our system into three main tiers: presentation, application and data tiers (better explained in Section 2.6) to better isolate common features between all tangible and intangible instances of all the objects we used. Below, a list of all the actors and objects involved:

- **Personal Device**: a personal device consists of a Sensor device (here modeled as the

union of all the device's movement sensors) and an OS that can be either Android or iOS. It is to be intended as a personal piece of hardware such as a smartphone or smartwatch that is carried by the user throughout the execution of the application. The personal device communicates with the Application Server using a local view instance of the app, here called Application: every request made by the Application to the Application Server follows a RESTful interaction style over HTTPS.

- **API Gateway**: it's a node offered by a third party (such as AWS or Google Cloud Services) that has a built-in WAF and load balancer to better distribute traffic to all the distributed Application Servers avoiding single points of failure.
- **Application Server**: it's a device that contains the logic of the application and connects together all the drivers needed to the application to work. It is connected with external APIs using REST and with multiple databases using a continuously working connection pool.
- **Database**: we decided to have different databases (one main database, more replicas used only in read mode) with a connection pool contained inside the Application Server, so that databases are actually protected by a limited amount of opened connection and overload is not possible. This feature is extremely useful to minimize concurrency problems and the number of connections can be adjusted accordingly to the number of requests seen to have the optimal balance of hardware usage and system efficiency: we chose Linux as the OS because of its open source roots and MySQL for its robustness.

Now for a closer look at some of the components of the devices themselves:

- **Application Artifact**: the client side application. It is used primary as the presentation layer and stores a signed JWT (JSON Web Token) used for authentication: it is connected with all sensors needed to detect movements in automated mode using cross-platform mobile frameworks.
- **BBP Server App**: the actual model of our application. Here, the logic of the app is stored and ensures all operations are coherent with user requests following the MVC pattern: it is the recipient of all the managers and connects them to every driver needed to interface to external tools.

2.4. Runtime View

2.5. Component Interfaces

2.6. Styles and Patterns

BBP will be developed using the client-server paradigm integrated with a three-tier architecture and the model-view-controller pattern: the aim of this mixed architecture is to achieve a clear separation between all the roles of the devices that are used and to better control each one of them. The tiers that are involved are:

- **The Presentation Tier** - This tier serves as the view component of the application and contains the software that needs to work strictly with the user's hardware (in our case, the personal device and the sensors connected to it): it can be seen as the top-level tier and it aims to be stateless, though some information about the user is saved by using JWT tokens for authentication. It's main functionality is to display all the informations and useful actions the user can do with the application itself and show all the outputs sent by the backend of the system, it does not contain the whole logic to prevent the user to change it locally and perform unauthorized actions: this also makes the application lighter in memory usage terms. To be able to actually make a platform-independent application compatible with the major operative systems we opted to use Flutter libraries in order to use a common language between OS in retrieving the sensors' informations.
- **The Application Tier** - This tier serves as the core logic component of the application and it is composed by the application servers, it is where all the user actions are checked and, if contextually correct with the state of the machine, validated and executed: it can be seen as a middle-level tier and as the space where all the data retrieved in the lower level are computed and then sent to the user in form of useful informations. We opted to have multiple application servers since with the use of an API Gateway (explained further) with a load balancer inside it is possible to do routing to specific servers, meaning that the same user will use the same server having the JWT as its id: this way, scaling is modular and possible and if a failure inside a server is detected it is possible to re-route the user to a different server avoiding downtime.
- **The Data Tier** - This tier serves as the main source of user and application data gathered inside databases: we opted to have a main database with a connection pool (so that the number of connections can actually be regulated granting flexibility and avoiding overloading) and several secondary databases used for read only operations, updated after write actions inside the main database. This achitecture avoids downtime of the data tier if the main database breaks (one of the secondary databases becomes the new main database) and at the same time avoids having cloned information issues (more than a main database can be difficult to handle by dealing with synchronization issues).

In addition to those three tiers, we opted to insert a third party API Gateway that connects the presentation tier to the application tier. To fully satisfy scalability and security concerns,

this API Gateway must have a load balancer (to evenly distribute clients connections between all servers) and a WAF (to protect attacks using a protection firewall): AWS and Google Cloud Services are some examples of third part companies that offer this kind of service.

As anticipated before the whole architecture can be seen as a client-server interaction, where all actions are event-driven: the client is the active part of almost all transactions (apart from notifications made by the server, like the starting of the automated mode recording) but is always checked by the logic inside the server, so that eventually communication errors or intentional attacks made by bad users are intercepted. Being the communication handled with REST it's easy to encode error messages and to use HTTPS to understand everything that is happening, both in front-end and back-end scenarios.

The main design pattern is the Model-View-Controller: the Model part is where the logic elements cohexist and work together to manage all the data to be elaborated, the View part contains the visual graphics shown to the user (using the software application

2.7. Other Design Decisions

3 | User Interface Design

4 | Requirements Traceability

5 Implementation, Integration and Test Plan

5.1. Overview and Implementation Plan

5.2. Features identification

5.3. Integration Strategy

5.4. System Testing Strategy

6 Effort Spent

A table representing the total hours of work is shown here.

Work Areas	Alessandro Burato	Gabriele Crimella	Davide Faccini
Introduction	1h	0h	0h
Architectural Design	8h	0h	0h
UI Design	0h	0h	0h
Requirements Traceability	0h	0h	0h
IIT	0h	0h	0h
Group Meetings and Reviews	0h	0h	0h

Table 6.1: Table of Effort

7 | References

List of Figures

2.1 Deployment View 3

List of Tables

6.1 Table of Effort 13